Universidad de San Carlos de Guatemala

Centro Universitario de Occidente CUNOC

División de las Ciencias de la Ingeniería

Teoria de Sistemas 1

Sección: A

Docente: Ivan Alejandro Esteban Archila

“Documentacion Final Clinica Dental”

Estudiantes:

Luis Alejandro Regalado Villatoro - 202131920
Carlos Alfredo López de León - 202231051
Mynor Miguel Monzón Martínez - 202230884
Herson Isaías Aguilar Juárez - 202230685
María Yohana Pérez López - 202030086

Quetzaltenango, 29 de octubre de

2026

1. Propósito

Sistema web que permite a una clínica dental registrar pacientes y personal, agendar citas con reglas de negocio, registrar tratamientos y planes de tratamiento con su costo y sus abonos, y consultar reportes, con un despliegue accesible desde internet.

2. Alcance

2.1 Qué incluye

- Registro e inicio de sesión con control de acceso por rol.
- Gestión de personal, pacientes, lista negra, historial clínico y horarios de atención.
- Agenda de citas con validación de horario, solapamiento y bloqueos de doctor, con cancelación y reprogramación sujetas a un plazo mínimo.
- Control de inasistencias de los pacientes con una forma controlada de levantar la restricción.
- Catálogo de tratamientos, registro de tratamientos por cita, planes de tratamiento de varias sesiones y abonos.
- Recordatorios dentro de la aplicación y por correo, y reportes con datos reales.
- Despliegue automatizado en AWS EC2 con integración y despliegue continuos.

2.2 Qué no incluye

- Odontograma.
- Pagos o facturación en línea.
- Notificaciones por mensaje de texto.
- Varias sedes.
- Aplicación móvil nativa.

3. Roles del sistema

El sistema distingue cuatro roles. Los permisos se validan en el backend y no solo en la interfaz.

Rol	Funciones principales
Administrador (ADMIN)	Gestiona usuarios y roles, el catálogo de tratamientos, los bloqueos de doctor, las restricciones por inasistencias, los abonos y los reportes, y puede operar sobre cualquier cita.
Secretaria o recepcionista (SECRETARIA)	Gestiona pacientes y la lista negra, agenda citas para los pacientes, registra horarios y bloqueos de doctor, levanta restricciones por inasistencias, registra abonos y consulta tratamientos y reportes.
Doctor u odontólogo (DOCTOR)	Administra sus horarios, consulta sus bloqueos, marca como atendidas o no asistió las citas propias, registra el historial clínico y los tratamientos de sus citas y crea planes de tratamiento.

Rol	Funciones principales
Paciente o cliente (CLIENTE)	Registra su cuenta, agenda, cancela y reprograma sus citas respetando el plazo, y consulta sus tratamientos y sus planes en solo lectura.

4. Requisitos funcionales

Cada requisito tiene un identificador (RF-xx) y se enlaza con su historia de usuario (HU) y con el número de issue del backlog en el tablero de GitHub Projects del repositorio TEO1Group/TEO1_ClinicaDental. Los criterios de aceptación completos están en el issue de cada historia.

ID	Requisito	Roles	Backlog
RF-01	Registro de cuenta de cliente. El sistema permite que una persona cree su cuenta con nombre, apellido, correo, teléfono, DPI, dirección, fecha de nacimiento y contraseña. Todos los campos son obligatorios y la contraseña exige mínimo 8 caracteres con mayúscula, minúscula, número y carácter especial. Si el DPI o el correo ya están registrados, se rechaza el registro con un mensaje de error.	Visitante (futuro paciente)	HU-01 (#10)
RF-02	Inicio de sesión. El sistema permite iniciar sesión con correo y contraseña y entrega un token JWT que se valida en cada petición. Ante credenciales incorrectas muestra un mensaje genérico, sin indicar qué campo falló. Cada usuario ve solo las opciones de su rol y un usuario desactivado no puede iniciar sesión.	Todos	HU-02 (#15)
RF-03	Gestión de personal y roles. Existe un administrador inicial configurado por variables de entorno. El administrador crea, lista, edita y activa o desactiva cuentas con rol DOCTOR, SECRETARIA o ADMIN; al crear un doctor se pide su especialidad y a una secretaria su turno y área. Cada rol usa solo las funciones que le corresponden y, si no tiene permiso, el sistema responde 403.	Administrador	HU-05 (#62)

ID	Requisito	Roles	Backlog
RF-04	Gestión de pacientes. Se registran, consultan, editan y desactivan pacientes desde el listado; al desactivar uno, su historial se conserva y deja de aparecer en el listado. Solo la secretaria agrega o quita pacientes de la lista negra, indicando el motivo.	Secretaria, Administrador	HU-06 (#67) HU-16 (#115)
RF-05	Historial clínico. El doctor y el administrador agregan registros al historial de un paciente, con la fecha puesta por el sistema; la secretaria solo lo consulta. El historial se muestra ordenado por fecha y, si no hay registros, se indica con un mensaje claro. El listado de pacientes se puede buscar por nombre o DPI.	Doctor, Administrador; Secretaria (consulta)	HU-04 (#24)
RF-06	Horarios de atención. Se registran, editan y eliminan los horarios de cada odontólogo por día de la semana, con hora de inicio y de fin. No se permiten dos horarios que se crucen el mismo día y un doctor solo modifica sus propios horarios.	Secretaria, Doctor	HU-08 (#76)
RF-07	Agendar cita. Se agenda eligiendo odontólogo, fecha y hora; el paciente agenda para sí mismo y la secretaria o el administrador eligen al paciente. La cita dura 30 minutos y queda en estado AGENDADA. Se rechaza con 409 si se solapa con otra cita no cancelada del mismo odontólogo o si no cabe en su horario, con 400 si la fecha ya pasó y con 403 si el paciente está en la lista negra. Una cita cancelada libera su horario.	Paciente, Secretaria, Administrador	HU-03 (#23; tareas #97 a #105)
RF-08	Estado de la cita. La cita maneja los estados AGENDADA, ATENDIDA, CANCELADA y NO_ASISTIO. Solo una cita AGENDADA puede cambiar de estado; los demás son finales y el sistema responde 409 si se intenta salir de ellos. El paciente cancela sus citas, el doctor marca las suyas como atendidas o no asistió, y la secretaria y el administrador pueden registrar cualquier estado final.	Paciente, Doctor, Secretaria, Administrador	HU-13 (#106)

ID	Requisito	Roles	Backlog
RF-09	Motivo y plazo al cancelar o marcar una cita. Cancelar o marcar NO_ASISTIO exige un motivo de máximo 200 caracteres; para ATENDIDA es opcional. El paciente solo cancela con 24 horas o más de anticipación (valor configurable); la secretaria y el administrador no tienen límite. El doctor no marca ATENDIDA ni NO_ASISTIO antes de la hora de inicio de la cita.	Paciente, Doctor, Secretaria, Administrador	HU-18 (#148)
RF-10	Reprogramar cita. Permite cambiar la fecha y la hora de una cita AGENDADA, que sigue AGENDADA. Se revalidan horario, solapamiento (sin contar la propia cita), fecha futura y bloqueos del doctor. El paciente solo reprograma la suya y con 24 horas o más de anticipación; la secretaria y el administrador no tienen límite. El motivo es obligatorio y el cambio queda registrado.	Paciente, Secretaria, Administrador	HU-19 (#152)
RF-11	Indisponibilidad del doctor por periodo. La secretaria y el administrador crean, listan y eliminan bloqueos de un doctor con inicio, fin y motivo; el doctor consulta los suyos. No se crea un bloqueo si el doctor ya tiene citas AGENDADAS en el periodo (409), y agendar o reprogramar dentro de un bloqueo se rechaza con 409 indicando el motivo.	Secretaria, Administrador; Doctor (consulta)	HU-20 (#155)
RF-12	Aviso de doctor no disponible. Al elegir un doctor en el formulario de cita o de reprogramación, el sistema muestra sus periodos de no disponibilidad vigentes con el motivo y, si la fecha y hora elegidas caen en un bloqueo, muestra una advertencia y deshabilita el envío. El backend mantiene el rechazo 409 como respaldo.	Paciente, Secretaria, Administrador	HU-31 (#219)

ID	Requisito	Roles	Backlog
RF-13	Recordatorio de citas próximas. Al iniciar sesión se muestra un aviso con la cantidad de citas AGENDADAS de las próximas 48 horas, y la campana del encabezado muestra un contador y un panel con esas citas. Aplica a pacientes y doctores, cada uno con las suyas; sin citas próximas el panel lo indica y no muestra contador.	Paciente, Doctor	HU-14 (#109)
RF-14	Aviso de cita por correo. Un día antes de la cita se envía un correo al paciente, sin repetir el mismo aviso; si la cita se reprograma, el aviso se vuelve a programar. Sin configuración SMTP el sistema y el CI funcionan igual.	Paciente (destinatario)	HU-15 (#112)
RF-15	Control de inasistencias. Cuando un paciente acumula 3 inasistencias (NO_ASISTIO, valor configurable), no puede agendar nuevas citas y el sistema responde 409 con un mensaje claro; la restricción aplica también al agendar sesiones de un plan. La secretaria y el administrador la levantan indicando un motivo obligatorio, queda registrado quién, cuándo y por qué, y el conteo vuelve a cero.	Secretaria, Administrador; Paciente (afectado)	HU-25 (#195)
RF-16	Catálogo de tratamientos. El administrador crea y edita tratamientos con nombre, descripción, duración y costo, y los desactiva en lugar de borrarlos; un tratamiento desactivado deja de ofrecerse. Cambiar el costo no altera los tratamientos ya registrados, que conservan el costo cobrado.	Administrador	HU-32 (#222)
RF-17	Registro de tratamientos por cita. El doctor registra los tratamientos realizados en una cita, con costo (por defecto el del catálogo, ajustable) y observaciones; solo el doctor de la cita y solo si está ATENDIDA. El historial del paciente muestra los tratamientos con fecha, doctor, costo y total, y el paciente consulta los suyos en solo lectura.	Doctor (registra); Administrador, Secretaria; Paciente (los suyos)	HU-21 (#158)

ID	Requisito	Roles	Backlog
RF-18	Planes de tratamiento de varias sesiones. Se crea un plan con paciente, doctor, tratamiento, número de sesiones y costo total. Las citas de cada sesión se vinculan al plan y respetan las reglas de agendado (horario, bloqueos e inasistencias). El avance es el cociente entre las sesiones atendidas y las sesiones totales.	Doctor, Secretaria, Administrador	HU-26 (#198)
RF-19	Abonos y saldo del plan. La secretaria y el administrador registran abonos de un plan; el saldo es el costo total menos los abonos y un abono no puede superar el saldo. El paciente consulta sus planes, su avance y su saldo en solo lectura.	Secretaria, Administrador; Paciente (consulta)	HU-26 (#198)
RF-20	Reporte de citas por doctor y estado. Muestra, para un periodo con filtro de fechas, las citas agendadas, atendidas, canceladas y no asistió de cada doctor, calculadas con datos reales de la base de datos.	Administrador, Secretaria	HU-22 (#161)
RF-21	Reportes finales. Además del anterior, el sistema ofrece los reportes de inasistencias y cancelaciones, ingresos por tratamiento, horas más demandadas y ganancias, todos con filtro de fechas y calculados con datos reales. Los reportes no cuentan las citas cuya hora de inicio aún no llega.	Administrador, Secretaria	HU-27 (#204)

5. Requisitos técnicos

Los requisitos técnicos se enlazan con el backlog y su detalle medible está en la sección 6.

ID	Requisito	Backlog
RT-01	Persistencia en PostgreSQL con esquema SQL y entidades alineadas con él.	HU-09 (#79)
RT-02	Dockerización: Dockerfile del backend y del frontend, y docker compose con base de datos, backend y frontend.	HU-10 (#83)
RT-03	Integración continua: compilación y pruebas del backend y compilación de producción del frontend en cada push y pull request.	HU-11 (#87)

ID	Requisito	Backlog
RT-04	Despliegue continuo en AWS EC2, con credenciales en GitHub Secrets y variables de entorno.	HU-12 (#90)
RT-05	Documentación de la API con Swagger: reglas de negocio, roles, errores y ejemplos de cada endpoint.	HU-23 (#164)
RT-06	Pipeline confiable y sprint desplegado con datos de demostración.	HU-24 (#169)
RT-07	Respaldo y restauración de la base de datos y datos de demostración.	HU-28 (#209)
RT-08	Flujo completo desplegado en EC2 con despliegue continuo activo.	HU-29 (#212)

6. Requisitos no funcionales

Este apartado reúne los requisitos no funcionales del sistema con una métrica verificable y los controles de seguridad básicos, cada uno con su evidencia en el código. Las referencias archivo:línea corresponden a la rama develop en el commit 6efcc50 (2026-10-29). Las mediciones se explican en la sección 6 para poder repetirlas.

No contiene ninguna contraseña, secreto ni valor del .env.

ID	Categoría	Requisito y métrica	Valor actual	Evidencia
RNF-01	Seguridad	La sesión (JWT) expira a las **24 horas** de iniciar sesión y se puede cambiar con JWT_EXPIRATION_MS.	86 400 000 ms = 24 h	application.properties:16, JwtService.java:58, JwtService.java:48, .env.example:11
RNF-02	Rendimiento	Con los datos de demo, el percentil 95 del tiempo de respuesta es **≤ 500 ms** en las consultas y **≤ 1 s** en el inicio de sesión.	Peor consulta: 9.0 ms. Login: 73.9 ms	Medición de la sección 6.2
RNF-03	Calidad	Las pruebas del backend cubren **al menos el 85 % de las líneas** y corren en el CI en cada PR y en cada push a develop y main que cambie código, base de datos o despliegue (filtro paths).	93.1 % de líneas, 78.7 % de ramas, 289 pruebas sin fallas	Sección 6.1, ci.yml:89, ci.yml:109

ID	Categoría	Requisito y métrica	Valor actual	Evidencia
RNF-04	Despliegue	Un cambio mergeado en main llega a EC2 **en 10 minutos o menos** y sin pasos manuales (CI más CD).	CI: mediana 2.8 min (máx. 5.0). CD: mediana 2.1 min (máx. 2.5). Último release: 6.2 min	Sección 6.3, cd.yml, deploy-ec2.sh
RNF-05	Disponibilidad	Si el backend o el frontend no responden **en 180 s** después de desplegar, el CD falla. Los contenedores se reinician solos si se caen o si se reinicia la EC2.	Ventana de 180 s; restart: unless-stopped en los 3 servicios	deploy-ec2.sh:77, deploy-ec2.sh:145-148, docker-compose.yml:4, docker-compose.yml:22, docker-compose.yml:49
RNF-06	Integridad	El backend **no arranca** si una entidad no coincide con el esquema de la base, y la base rechaza valores fuera de rango con 20 restricciones CHECK.	ddl-auto=validate	application.properties:11, clinica_dental_schema.sql (por ejemplo, línea 216 y línea 224)
RNF-07	Concurrencia	**Nunca** quedan dos citas activas del mismo doctor en la misma fecha y hora, aunque se agenden al mismo tiempo.	Índice único parcial y bloqueo pesimista del doctor	clinica_dental_schema.sql:163-164, DoctorRepository.java:27-29, CitaService.java:300, CitasConcurrencyIntegrationTests
RNF-08	Exactitud	Las fechas y horas de negocio (citas, plazos de 24 h, recordatorios) se calculan en la zona **America/Guatemala**, sin depender de la zona del servidor.	app.zona-horaria=America/Guatemala	application.properties:29, ClockConfig.java:14

ID	Categoría	Requisito y métrica	Valor actual	Evidencia
RNF-09	Recuperación	Cada respaldo se **verifica al crearlo** y se puede restaurar en una base aparte comparando las filas de cada tabla.	Probado en EC2 el 2026-10-21: 16 tablas con las mismas filas	respaldar-db.sh:36, respaldar-db.sh:22, restaurar-db.sh, documento docs/respaldo-y-restauracion.md
RNF-10	Mantenibilidad	Las migraciones se pueden **correr más de una vez** sin error ni cambios extra.	Las 3 del Sprint 6 se aplicaron dos veces en EC2 el 2026-10-28	2026-10-sprint6-planes.sql:13 (IF NOT EXISTS) y las demás de db/migrations/
RNF-11	Portabilidad	Todo el sistema (PostgreSQL, backend y frontend) se levanta con **un solo comando**, docker compose up.	3 servicios en Compose	docker-compose.yml
RNF-12	Documentación	**Todas** las operaciones de la API están en Swagger, con respuestas y errores; una prueba falla si falta alguna.	100 % de las operaciones de los controllers	OpenApiDocumentationIntegrationTests (everyControllerOperationDocumentsSecurityAndResponseSchemaPresence)

6.1 Cobertura de pruebas (RNF-03)

Se midió sobre develop (6efcc50) con JaCoCo 0.8.12 desde la línea de comandos, sin cambiar el pom.xml, contra PostgreSQL 16 con db/clinica_dental_schema.sql cargado. Desde backend/, con las variables de la base y JWT_SECRET exportadas:

```
./mvnw -B clean org.jacoco:jacoco-maven-plugin:0.8.12:prepare-agent verify
org.jacoco:jacoco-maven-plugin:0.8.12:report
```

El reporte queda en backend/target/site/jacoco/index.html.

Medida	Cubiertas / total	%
Líneas	1 261 / 1 354	93.1 %
Ramas	377 / 479	78.7 %
Instrucciones	6 603 / 7 162	92.2 %
Métodos	281 / 318	88.4 %

Por paquete, en líneas:

Paquete	Líneas cubiertas	%
service	961 / 1 036	92.8 %
config	110 / 110	100 %
controller	69 / 79	87.3 %
security	53 / 57	93.0 %
model	37 / 37	100 %
exception	19 / 21	90.5 %
dto	11 / 11	100 %

En total corrieron 289 pruebas en 33 clases, con 0 fallas, 0 errores y 0 omitidas.

6.2 Tiempo de respuesta (RNF-02)

Condiciones de la medición:

- **Backend:** el JAR de develop (6efcc50) con la configuración de producción (incluido show-sql), en una laptop Intel Core i5-1135G7 con 8 GB de RAM.
- **Base:** PostgreSQL 16 en local con los datos de db/demo/datos-demo.sql.
- **Procedimiento:** cada endpoint recibió 20 peticiones de calentamiento y luego 200 medidas, en secuencia, todas con respuesta 200.

Endpoint	p50 (ms)	p95 (ms)	Máx. (ms)
POST /auth/login	70.8	73.9	82.9
GET /citas (secretaria)	6.8	9.0	17.4
GET /citas (doctor)	6.2	7.7	11.2
GET /pacientes	4.1	5.1	15.7
GET /doctores	3.7	5.0	8.2
GET /planes	5.4	7.1	10.2
GET /planes/{id}	6.2	7.3	17.1
GET /reportes/ganancias (60 días)	4.6	5.6	6.6
GET /reportes/ingresos-por-tratamiento (60 días)	3.1	4.0	6.2
GET /reportes/inasistencias (60 días)	3.3	4.3	7.3

El login es el más lento a propósito: BCrypt está diseñado para que calcular el hash cueste tiempo, y eso hace más caro un ataque de fuerza bruta.

6.3 Tiempo de despliegue (RNF-04)

Los tiempos se sacaron de los registros de GitHub Actions (gh run list y gh run view --json jobs). Para el CD se sumó la duración de sus dos jobs (build-and-push y deploy). No se usó la duración total de la corrida porque en algunos releases la EC2 estaba apagada (Learner Lab) y el job se reejecutó más tarde, lo que inflaría el tiempo.

- **CI:** 39 corridas exitosas recientes, con una mediana de 2.8 minutos y un máximo de 5.0.
- **CD:** 8 releases exitosos, con una mediana de 2.1 minutos y un máximo de 2.5. El job deploy tarda entre 0.2 y 0.7 minutos.
- **Último release limpio:** el #244 (dc45780, 2026-10-28) tardó 6.2 minutos desde que empezó el CI hasta que terminó el CD (CI run 40724695227, CD run 40725278005).

7. Restricciones y supuestos

7.1 Restricciones

- Los valores de negocio fueron acordados por el equipo como Product Owner: plazo de 24 horas para cancelar o reprogramar, límite de 3 inasistencias y motivo de máximo 200 caracteres.
- Las citas duran 30 minutos fijos y se manejan en hora de Guatemala.
- Un tratamiento solo se registra sobre una cita en estado ATENDIDA y lo registra el doctor de esa cita.
- La aplicación no permite crear citas con fecha y hora pasadas.

- Tecnología: Java 21, Spring Boot 4.1.1, Angular 22, PostgreSQL 16, Docker, GitHub Actions y AWS EC2.
- Las migraciones de la base de datos se aplican manualmente en EC2 antes de cada release; el despliegue continuo no las ejecuta.
- Ninguna credencial se guarda en el repositorio: se manejan con GitHub Secrets y variables de entorno.

7.2 Supuestos

- La clínica opera en una sola sede.
- Cada doctor tiene su horario semanal cargado antes de recibir citas.
- Cada paciente tiene una sola cuenta, identificada por un DPI y un correo únicos.
- Los correos de recordatorio dependen de que el servicio SMTP esté configurado en el servidor.
- Los costos de los tratamientos del catálogo los define el administrador, y el doctor puede ajustarlos al registrar el tratamiento.

8. Seguridad básica

8.1 Contraseñas guardadas con hash BCrypt

- Las contraseñas se guardan con BCrypt (BCryptPasswordEncoder, costo 10 por defecto): SecurityConfig.java:39.
- El hash se calcula en los tres lugares donde se crean usuarios:
  - Registro de pacientes: AuthService.java:61.
  - Alta de personal: AdminUsuarioService.java:115.
  - Administrador inicial: AdminInitializer.java:68.
- Al iniciar sesión se compara contra el hash; nunca se descifra: AuthService.java:89.
- Si el correo no existe o la contraseña es incorrecta, la respuesta es la misma ("Email o contraseña incorrectos"), así que no revela qué correos están registrados: AuthService.java:83-93.
- Las contraseñas deben tener al menos 8 caracteres, con mayúscula, minúscula, número y símbolo: RegistroRequest.java:42-46 y CrearPersonalRequest.java:35-40.
- La contraseña nunca se devuelve:
  - Las respuestas de registro solo traen mensaje e id: RegistroResponse.java:14-16.
  - En Swagger el campo es de solo escritura: RegistroRequest.java:47.
- Los datos de demo también guardan la contraseña con BCrypt (crypt(..., gen_salt('bf', 10)) de pgcrypto): datos-demo.sql:85; la contraseña se pasa al ejecutar el script: datos-demo.sql:40-44.

8.2 Expiración y validez de la sesión

- El token se firma con HMAC-SHA y una llave derivada de JWT_SECRET: JwtService.java:25 y JwtService.java:65. La firma se verifica en cada petición: JwtService.java:70-73.
- La fecha de expiración se pone al emitir el token (JwtService.java:58 y JwtService.java:64), y un token vencido o sin rol válido se rechaza (JwtService.java:44-50). Duración: RNF-01.
- Un usuario desactivado pierde el acceso aunque tenga un token vigente:
  - El filtro comprueba en cada petición que el usuario siga ACTIVO: JwtAuthenticationFilter.java:50 y JwtAuthenticationFilter.java:71-73.
  - El login de una cuenta inactiva devuelve 403: AuthService.java:96-97.
- El servidor no guarda sesiones (STATELESS): SecurityConfig.java:48.

8.3 Permisos por rol validados en el backend

Los permisos se validan en dos capas en el servidor, sin depender de lo que muestre el frontend.

1. **Por ruta y método HTTP**, en SecurityConfig: SecurityConfig.java:56-101.
   - Solo son públicos el registro, el login, Swagger y la versión: SecurityConfig.java:59-61.
   - Cualquier ruta no listada exige sesión: SecurityConfig.java:100.
   - Sin sesión se responde 401 y con un rol sin permiso, 403: SecurityConfig.java:52-55.
2. **Por registro**, en los servicios. Hay 18 rechazos con 403 en 7 servicios, entre ellos los que dependen de a quién pertenece el dato. Ejemplos:
   - Entre doctores y pacientes, solo el doctor de la cita la marca atendida o no asistió y solo el paciente dueño la cancela; secretaria y administrador pueden con cualquiera: CitaService.java:182-194.
   - Los doctores no agendan citas: CitaService.java:275.
   - Solo el doctor de la cita registra tratamientos: TratamientoService.java:95.
   - Un doctor solo crea planes para sí mismo y solo ve los suyos: PlanTratamientoService.java:288 y PlanTratamientoService.java:302.

Pruebas: SeguridadSprint5IntegrationTests, SeguridadSprint6IntegrationTests y FlujoCompletoIntegrationTests, que recorre el flujo con login real de los tres roles del personal.

8.4 Validación de entradas

- Los cuerpos de las peticiones se validan con Bean Validation (@Valid, 23 usos en los controllers; por ejemplo, AuthController.java:43). Algunas reglas:
  - El DPI debe tener 13 dígitos: RegistroRequest.java:21.
  - El correo debe tener formato válido: RegistroRequest.java:36.

    - El teléfono debe tener 8 dígitos: CrearPersonalRequest.java:43.
- Un dato inválido devuelve 400 con el error de cada campo, sin llegar al servicio: GlobalExceptionHandler.java:29-41.
- Un JSON mal formado o un parámetro de tipo incorrecto (por ejemplo, un UUID inválido) también devuelven 400: GlobalExceptionHandler.java:43-51.
- Como segunda capa, la base rechaza valores fuera de rango con CHECK (RNF-06). Por ejemplo, un abono debe ser mayor que 0 (clinica_dental_schema.sql:216).

8.5 Consultas parametrizadas

- Todo el acceso a datos pasa por Spring Data JPA:
    - Consultas derivadas del nombre del método.
    - Las 11 @Query, en JPQL con parámetros con nombre (:desde, :hasta, :id…); por ejemplo, ReporteCitaRepository.java:17-19 y DoctorRepository.java:28-29.
    - Ningún valor del usuario se concatena en el texto de una consulta.
- No hay SQL nativo ni SQL armado con texto en el backend. La búsqueda de nativeQuery, createNativeQuery, createQuery, JdbcTemplate y concatenaciones de SELECT en backend/src/main no encuentra nada.
- El log de Hibernate (application.properties:12) muestra el SQL con ? en lugar de los valores, por ejemplo where re1_0.nombre_rol=?, porque los valores van como parámetros.

8.6 Credenciales fuera del repositorio

- El backend lee las credenciales de variables de entorno, sin valores por defecto:
    - Base de datos: application.properties:6.
    - JWT: application.properties:14.
    - Administrador inicial: application.properties:23.
    - SMTP: application.properties:40.
- docker-compose.yml también las toma del entorno: docker-compose.yml:8 y docker-compose.yml:29-30.
- El .env real no se versiona: .gitignore:18-20. Nunca se agregó al historial (git log --all -- .env no devuelve nada).
- La plantilla .env.example deja vacíos los valores sensibles: .env.example:3, .env.example:8, .env.example:15 y .env.example:21.
- El CI y el CD usan GitHub Secrets y el CI falla si faltan: ci.yml:57-58 y cd.yml:100-102.
- Los scripts de operación no piden ni guardan contraseñas:
    - El de respaldo usa las del contenedor: respaldar-db.sh:9.
    - El de datos de demo recibe la suya al ejecutarse: datos-demo.sql:40-44.

- No hay llaves (.pem, .key, .p12, .jks) versionadas. Una búsqueda de asignaciones de contraseñas o secretos con valor en los archivos versionados no encuentra ninguna.

8.7 Otros controles

- CORS solo admite los orígenes de CORS_ALLOWED_ORIGINS (en EC2, el frontend en el puerto 4200): SecurityConfig.java:124, application.properties:19 y cd.yml:96.
- Cada FK que registra quién hizo una acción (reprogramaciones, bloqueos, levantamientos de inasistencias y abonos) guarda el usuario que la hizo; ver el diagrama ER de la sección 9.

8.8 Limitaciones conocidas

Son decisiones o pendientes que el equipo conoce, para que el documento no afirme más de lo que hay.

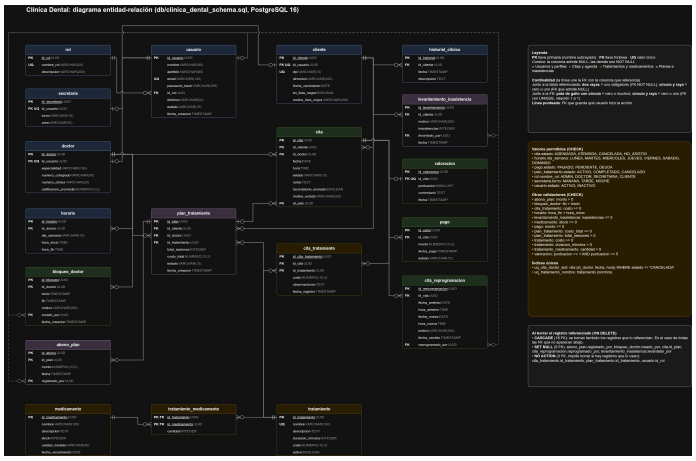
Tema	Situación	Evidencia
HTTPS	EC2 sirve el frontend y la API por HTTP (puertos 4200 y 8080), sin TLS.	cd.yml:96
Almacenamiento del token	El frontend guarda el JWT en localStorage. Un ataque XSS podría leerlo.	auth.service.ts:54
CSRF	Está desactivado porque la API no usa cookies de sesión: el token viaja en el encabezado Authorization.	SecurityConfig.java:46, auth.interceptor.ts:18
Intentos de login	No hay límite de intentos ni bloqueo temporal por contraseña incorrecta.	AuthService.java:83-93
Log de SQL	spring.jpa.show-sql=true también está activo en producción. Muestra el SQL sin los valores de los parámetros.	application.properties:12
Volumen de datos	Los tiempos de RNF-02 se midieron con los datos de demo (8 usuarios y unas 25 citas), no con volumen de producción.	Sección 6.2

9. Diagramas

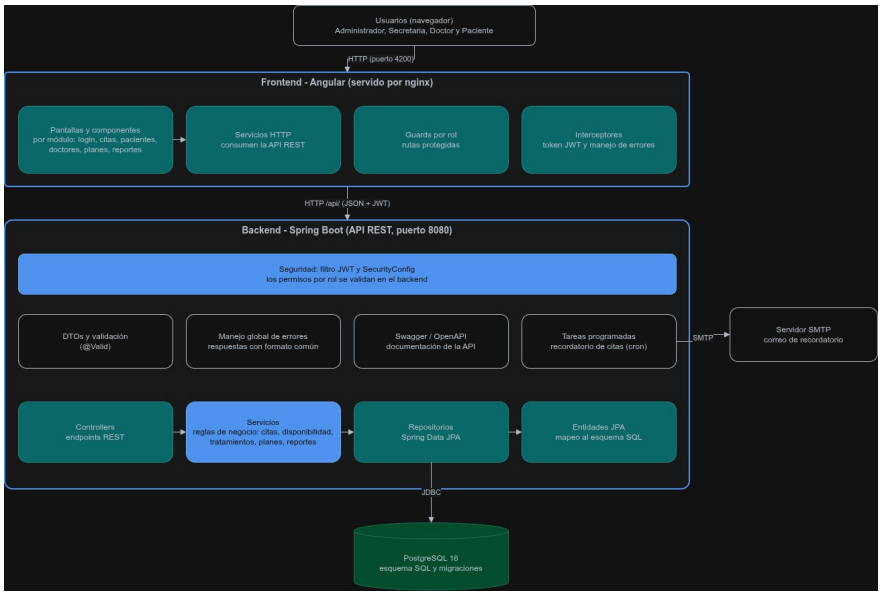
Los diagramas se mantienen en formato editable (.drawio) y como imagen (.png) en la carpeta docs/diagramas del repositorio.

9.1 Diagrama entidad-relación

Representa las tablas de la base de datos clinica_dental y sus relaciones, según db/clinica_dental_schema.sql.



9.2 Arquitectura del sistemaMuestra las capas del sistema: el navegador, el frontend Angular servido por nginx, el backend Spring Boot organizado en capas y la base de datos PostgreSQL, junto con las tareas programadas y el servidor de correo.



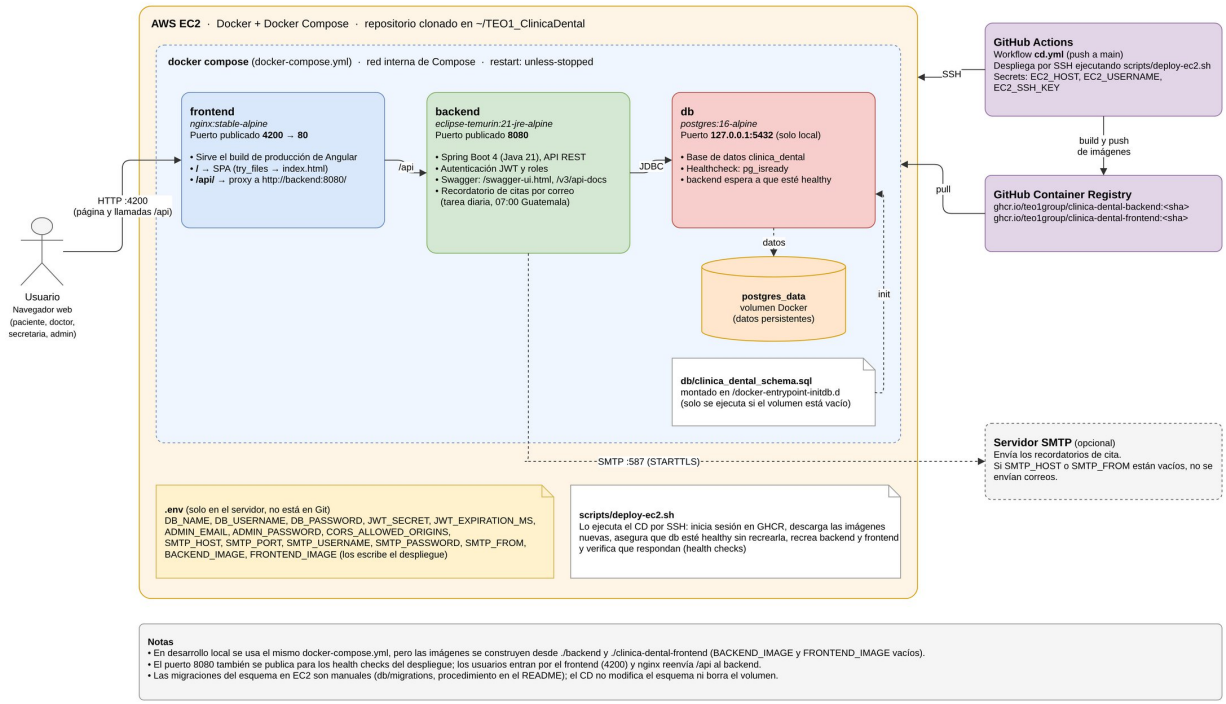
9.3 Despliegue en AWS EC2

Muestra el flujo desde GitHub hasta la instancia: la integración continua, el despliegue continuo, el registro de imágenes GHCR y los tres contenedores que corren en EC2.

9.4 Procesos del flujo principal

Describe el flujo de registro, agendamiento con sus validaciones, atención y registro de tratamientos, con las ramas de cancelación, reprogramación, bloqueo de doctor e inasistencias.

Clínica Dental: diagrama de despliegue (producción en AWS EC2)



10. Historial de sprints y retrospectiva

Sistema de gestión para clínica dental

Teoría de Sistemas 1 - Proyecto 2, Grupo 2

10.1 Historial de sprints

El proyecto se organizó en los sprints 0, 2, 3, 4, 5, 6 y 7: no hubo un Sprint 1 con pauta de evaluación y se pasó directamente del Sprint 0 al 2. La tabla muestra el período de cada uno según los releases de develop a main; las fechas oficiales de cada revisión pueden diferir unos días. En total se fusionaron 82 pull requests de trabajo y 9 releases de develop a main, y los problemas se registran en cada sprint con la forma en que se resolvieron.

Sprint	Período	Release a main
Sprint 0	6 de agosto	Ficha de arranque, repositorio y tablero
Sprint 2	20 de agosto al 23 de septiembre	#31 (10 sep, revertido y reintegrado en #49) y #49 (23 sep)
Sprint 3	22 al 24 de septiembre	#58 y #96 (24 sep)
Sprint 4	24 de septiembre al 8 de octubre	#138 (8 oct) y #145 (9 oct)
Sprint 5	9 al 16 de octubre	#189 (15 oct) y #194 (16 oct)
Sprint 6	16 al 28 de octubre	#244 (28 oct)
Sprint 7	Desde el 28 de octubre	En curso

Sprint 0 - Arranque

Qué se entregó. La ficha de arranque del proyecto (grupo 2, empresa Clínica Dental, stack con Angular, Spring Boot, PostgreSQL y Docker Compose, paleta de colores y tipografía), el repositorio con su README y el tablero de GitHub Projects. En el primer día se hicieron pull requests de práctica con ramas y conflictos de Git (#5 a #8).

Cambios del Product Owner. Las respuestas anotadas en la ficha definieron las entidades, los roles (administrador, secretaria, doctor y cliente), las reglas iniciales (sin traslapes de horarios ni citas, lista negra, recordatorios de citas) y los reportes esperados (inasistencias, cancelaciones, historial y deudas).

Problemas y cómo se resolvieron. La cuenta de AWS, la instancia EC2 y la llave SSH quedaron pendientes, y Docker y Node no estaban instalados en todos los equipos; se dejaron como tareas de los sprints siguientes. La asignación de tareas fue lenta al inicio, y se corrigió al crear las historias de usuario con sus sub-tareas y responsables en el tablero.

Sprint 2 - Primeros incrementos

Pauta. Autenticación y roles (0.5 pts), gestión de pacientes (0.4), odontólogos y disponibilidad (0.4), persistencia real en base de datos (0.5) y buenas prácticas de Git (0.2).

Qué se entregó. Estructura inicial de Angular (#20), registro de cuenta con JWT en el backend (#21) y su formulario (#22), inicio de sesión (#32 y #27), rutas y guards por rol (#29), registro de personal (#33) y el esquema de la base de datos (#31 y #34). Después se agregaron el listado y la gestión de usuarios, el administrador inicial configurado por variables de entorno (#46), el CRUD de pacientes y doctores (#48), las funciones de médicos y horarios (#47) y las entidades alineadas con el esquema SQL (#37). El release #49 del 23 de septiembre integró 118 archivos. Corresponde a las historias HU-01, HU-02, HU-04, HU-05, HU-06, HU-08 y HU-09.

Cambios del Product Owner. Ninguno registrado en este sprint.

Problemas y cómo se resolvieron. El primer release a main (#31) entró directo desde una rama de funcionalidad, sin pasar por develop, y se revirtió cuatro minutos después; su contenido volvió a main en el release #49. Además, al integrar se borraron archivos por error y hubo que restaurarlos en un pull request aparte (#50, con su release #52 el mismo día). Desde entonces el flujo es rama de funcionalidad, pull request a develop y release de develop a main, y se revisa el listado de cambios antes de integrar.

Sprint 3 - Automatización (CI y CD)

Pauta. Dockerización (0.4 pts), despliegue en AWS EC2 (0.4), integración continua con compilación y pruebas (0.5), despliegue continuo automático a EC2 (0.5) y buenas prácticas con credenciales en secrets (0.2).

Qué se entregó. Dockerfile y nginx del frontend y docker compose con base de datos, backend y frontend (HU-10), flujo de CI en cada push y pull request (HU-11) y flujo de CD con imágenes en GHCR y despliegue por SSH a EC2 (HU-12), con actualización automática de CORS. Pull requests #41 a #45 y #57; release #58 del 24 de septiembre y una prueba del CI en #95 y #96.

Cambios del Product Owner. Ninguno registrado en este sprint.

Problemas y cómo se resolvieron. La configuración de AWS y EC2 fue difícil para el equipo y requirió varias correcciones seguidas de despliegue, persistencia de imágenes, CORS y producción (#43, #44, #45 y #57). Se resolvió con correcciones pequeñas y frecuentes hasta que el despliegue automático funcionó sin pasos manuales.

Sprint 4 - Incremento de funcionalidades

Pauta. Agendamiento de citas (0.9 pts), recordatorio de citas dentro de la aplicación (0.6), pruebas automatizadas de la validación de solapamiento en el CI (0.6), documentación Swagger (0.5) y buenas prácticas con el pipeline funcionando (0.4). El aviso por correo era un bono de 0.10 puntos.

Qué se entregó. Agendar citas con validación de solapamiento, horario del doctor, fecha pasada y lista negra (HU-03); los cuatro estados de la cita (HU-13); recordatorios con aviso y campana (HU-14); aviso por correo (HU-15); registro y desactivación de pacientes desde la pantalla (HU-

16); pruebas automatizadas y documentación Swagger. Release #138 del 8 de octubre. Después del release se ajustaron los permisos de estado y de recordatorios (#141), el README (#142), el inicio de sesión (#143) y los diagramas (#144), con el release #145.

Cambios del Product Owner. Se agregó la documentación Swagger como criterio de evaluación, que no estaba en la planificación oficial, y el bono del correo. En la revisión se pidió que Swagger describa el backend con sus reglas y errores, y que todo cambio de estado de una cita tenga un motivo.

Problemas y cómo se resolvieron. El envío de correos con SMTP no quedó bien implementado al cierre del sprint, y se retomó en el Sprint 5 con la configuración y el horario del recordatorio en variables de entorno (#184). El inicio de sesión con credenciales inválidas mostraba un mensaje equivocado y dejaba el botón cargando, y se corrigió en #143. Además, 13 pull requests se integraron en una sola noche, por lo que la revisión fue apresurada.

Sprint 5 - Consolidación

Pauta. Registro de tratamientos con costo en el historial del paciente (0.6 pts), cambios del Product Owner sobre el plazo de cancelación y el bloqueo de horarios (0.5), reportes con datos reales (0.3), pruebas automatizadas de las reglas nuevas (0.4) y Git, credenciales y Swagger actualizado (0.2).

Qué se entregó. Motivo obligatorio y plazo de 24 horas para cancelar (HU-18), reprogramar cita (HU-19), bloqueos de doctor por periodo (HU-20), tratamientos por cita con costo y total por paciente (HU-21) y la pantalla de tratamientos del paciente (#191 y #193), reporte de citas por doctor (HU-22), Swagger completo con una prueba que falla si falta documentar un endpoint (HU-23), CD condicionado al CI, corrección de consultas N+1, datos de demostración y README. Releases #189 del 15 de octubre y #194 del 16 de octubre.

Cambios del Product Owner. Las citas agendadas dejaron de poder modificarse o cancelarse libremente, y se acordó un plazo mínimo de 24 horas; y la clínica pudo marcar que un doctor no está disponible durante un periodo, que el sistema respeta al agendar y reprogramar. Los valores concretos los propuso y documentó el equipo.

Problemas y cómo se resolvieron. Las tareas dependían del esquema de la base de datos, por lo que se fusionó primero en un pull request aparte y las demás avanzaron después. Al fusionar las ramas del frontend aparecieron conflictos y una clase duplicada que rompió el CI, y se resolvieron a mano y con una corrección posterior. La migración de la base en EC2 debe aplicarse manualmente antes de cada release, porque el backend valida el esquema al arrancar, y se dejó como paso explícito del procedimiento.

Sprint 6 - Cierre funcional

Pauta. Reportes finales (0.3 pts), ajustes finales del Product Owner (0.7), flujo completo desplegado con CD activo (0.4), documentación con manual de usuario y manual técnico (0.4) y datos de demostración con respaldo de la base (0.2).

Qué se entregó. Control de inasistencias con bloqueo al agendar y levantamiento con motivo (HU-25), planes de tratamiento de varias sesiones con citas vinculadas, avance y abonos (HU-26), reportes de inasistencias y cancelaciones, ingresos por tratamiento, horas más demandadas y ganancias (HU-27), catálogo de tratamientos administrable (HU-32), aviso de doctor no disponible (HU-31), respaldo y restauración de la base y datos de demostración (HU-28), y auditoría de la documentación Swagger. Release #244 del 28 de octubre, con 110 archivos.

Cambios del Product Owner. El control de inasistencias de los pacientes, con una consecuencia al reincidir al agendar y una forma controlada de levantarla; y los tratamientos que requieren varias sesiones, con seguimiento de su avance y de su costo con abonos.

Problemas y cómo se resolvieron. Entre el 16 y el 20 de octubre no hubo actividad en el repositorio, y 12 pull requests se integraron entre el 20 y el 22 de octubre. Las pantallas de planes y reportes llegaron al final del sprint y la documentación quedó pendiente; se resolvió repartiendo de nuevo las tareas según la carga y dejando los manuales y el documento final para el Sprint 7.

Sprint 7 - Entrega final

En curso. Incluye el documento final, el historial y la retrospectiva, la tabla de estimaciones contra tiempo real, los manuales de usuario y técnico, el cierre del frontend, las pruebas de punta a punta y la preparación de la presentación.

10.2 Retrospectiva

10.2.1 Qué funcionó

- Un flujo de ramas con pull requests: salvo el commit inicial y una reversión, todos los cambios entraron por pull request, con una aprobación de otro integrante para llegar a main.
- Entregar el contrato de cada endpoint al frontend apenas se terminaba la fase de backend, sin esperar la fusión, para que ambos equipos avanzaran en paralelo.
- Separar el esquema de la base de datos en su propio pull request y usar migraciones que se pueden ejecutar varias veces.
- Concentrar las reglas de disponibilidad de una cita en un solo servicio, que usan al agendar, reprogramar y validar bloqueos, lo que evitó reglas duplicadas.
- Pruebas automatizadas con caso exitoso y de rechazo para cada regla nueva, que se ejecutan en el CI, y una prueba que falla si un endpoint queda sin documentar en Swagger.

10.2.2 Qué no funcionó

- La preparación tardía de la infraestructura, que consumió el Sprint 3.
- Las entregas concentradas al final de cada sprint, con revisiones apresuradas.
- La documentación y las estimaciones de tiempo se dejaron para el final, y no se registraron horas reales.

- Los pagos de los pacientes se resolvieron con el costo de los tratamientos y los abonos de los planes de tratamiento.

10.2.3 Lecciones aprendidas y acciones concretas

Cada problema observado se acompaña de lo que el equipo haría distinto en un próximo proyecto.

Problema observado	**Qué haríamos distinto**
La asignación de tareas fue lenta al inicio del proyecto.	Repartir las tareas el primer día de cada sprint, con responsable y fecha de entrega en el tablero, en una reunión de planificación de una hora.
La cuenta de AWS, la instancia EC2 y la llave SSH no estaban listas en el Sprint 0, y configurarlas costó varias correcciones seguidas en el Sprint 3.	Preparar la infraestructura (AWS, EC2, llaves, Docker y Node en todos los equipos) durante el Sprint 0 y probar un despliegue vacío antes de escribir funcionalidades.
Las entregas se concentraron en una sola noche: 13 pull requests el 8 de octubre, 10 el 15 de octubre y 12 entre el 20 y el 22 de octubre.	Integrar a develop cada uno o dos días con pull requests pequeños, y fijar un hito intermedio en la mitad de cada sprint.
Entre el 16 y el 20 de octubre no hubo actividad en el repositorio, y las pantallas de planes y reportes llegaron al final del Sprint 6.	Acordar por escrito el contrato de la API al abrir cada historia, para que frontend y backend avancen en paralelo desde el primer día.
El envío de correos con SMTP no quedó bien resuelto en el Sprint 4.	Probar las integraciones externas con una cuenta real al inicio del sprint y no al final, y dejar su configuración en variables de entorno desde la primera versión.
El primer release a main entró directo desde una rama de funcionalidad y se revirtió a los pocos minutos; después hubo que restaurar archivos borrados.	Activar la protección de la rama main desde el primer día, integrar solo a través de develop y revisar el listado de archivos de cada pull request antes de aprobarlo.
Se resolvieron conflictos de fusión entre ramas del frontend, y una fusión dejó una clase duplicada que rompió el CI.	Hacer pull de develop antes de abrir cada pull request, repartir los archivos compartidos entre pocas personas y confiar en el CI para detectar el error antes de aprobar.
Los criterios de aceptación de algunas historias quedaron desactualizados respecto al sistema, y la documentación se dejó para el final.	Actualizar el criterio de la historia cuando cambia la regla, y escribir un fragmento de documentación al cerrar cada historia.
No se registraron horas reales por tarea ni estimaciones en el repositorio, por lo que fue difícil comparar lo estimado con lo real.	Registrar la estimación y las horas reales de cada issue en el tablero desde el primer sprint.

10.2.4 Reparto de roles

Los roles iniciales salen de la ficha de arranque. La columna de trabajo real resume lo que cada integrante hizo según el historial de pull requests.

Integrante	Rol inicial	Trabajo real
Luis Alejandro Regalado Villatoro	Base de datos y backend	Esquema y migraciones, inasistencias, planes de tratamiento, reportes, datos de demostración, respaldo, corrección de consultas N+1 y versión desplegada.
Carlos Alfredo López de León	Backend	Infraestructura: Docker, CI y CD, despliegue en EC2, documentación Swagger, abonos, catálogo de tratamientos y reporte de horas demandadas.
Mynor Miguel Monzón Martínez	Scrum Master y backend	Backlog y coordinación del equipo; autenticación, citas y sus reglas de negocio (agendar, estados, motivo y plazo, reprogramar, bloqueos), tratamientos y reporte por doctor. En los sprints 6 y 7, documentación.
Herson Isaias Aguilar Juárez	Frontend	Estructura Angular, formularios, rutas y guards, citas, bloqueos, tratamientos y la pantalla de tratamientos del paciente.
María Yohana Pérez López	Frontend	Pantalla de login, pacientes, historial y tratamientos, reportes y planes de tratamiento.

11. Estimaciones de tiempo y comparativa con el desarrollo real

11.1 Metodología

Este apartado compara, por tarea, la estimación de tiempo con las horas reales aproximadas que reportó cada integrante. Cada tarea se enlaza con el identificador del requisito de este documento (RF para los funcionales, RT para los técnicos, RNF para los no funcionales y Doc. para las tareas de documentación) y con su issue del tablero de GitHub Projects.

- **Estimación probable:** las horas que el Scrum Master calculó al planificar cada sprint, tomadas de los archivos de tareas de los sprints 5, 6 y 7. Para los sprints 2 a 4 no se conservó una estimación por tarea, por lo que en esos sprints solo se presentan las horas reportadas y no se calcula desviación.
- **Estimaciones optimista y pesimista:** no se registraron en su momento. Se reconstruyen con una regla fija: optimista = 70 % de la probable y pesimista = 150 % de la probable, redondeadas a 0.25 h. Son una reconstrucción y no una estimación hecha durante el sprint.

- **Tiempo real:** horas aproximadas reportadas por cada integrante por issue, que incluyen desarrollo, revisión, pruebas e integración de los cambios. El tablero no llevó un registro de horas. La sigla n/r indica que el integrante no reportó sus horas.
- **Apoyo:** indica, según lo que informó cada integrante, si la tarea se hizo de forma manual, mixta (con un porcentaje de apoyo de IA cuando lo indicó) o con apoyo de IA.
- **Desviación:** horas reales menos horas de la estimación probable. Un valor positivo indica que la tarea tomó más de lo estimado.

Limitaciones. Las horas son aproximadas y no se pueden verificar con una herramienta de medición. La evidencia de git no sirve para medir horas: la mayoría de los pull requests tiene un solo commit hecho minutos antes del merge, por lo que solo se usa para ubicar la fecha de entrega. Las tareas sin horas reportadas no entran en los totales.

11.2 Resumen por sprint

La tabla resume las tareas que tienen estimación probable y horas reales reportadas.

Sprint	Tareas en la tabla	Tareas comparables	Probable (h)	Real (h)	Desv. (h)	Desv. %
Sprints 2 y 3	7	0	—	—	—	—
Sprint 4	22	0	—	—	—	—
Sprint 5	22	22	61.5	60.75	-0.75	-1 %
Sprint 6	18	16	51	37	-14	-27 %
Sprint 7 (tareas cerradas)	8	8	24.5	27.5	+3	+12 %

Comparativa por tipo de trabajo en los sprints 5 a 7 (46 tareas comparables):

Tipo de trabajo	Tareas	Probable (h)	Real (h)	Desv. (h)	Desv. %
Backend e infraestructura	25	74	43.25	-30.75	-42 %
Frontend	16	49	69.5	+20.5	+42 %
Documentación	5	14	12.5	-1.5	-11 %

11.3 Detalle por sprint

Optimista, probable y pesimista están en horas. La columna Causa explica la desviación o, en los sprints sin estimación, lo que tomó el tiempo.

11.3.1 Sprints 2 y 3 (20 de agosto al 24 de septiembre de 2026)

Req.	Issue	Tarea	Resp.	Opt	Prob	Pes	Real	Desv	Apoyo	Causa
RF-01	#13	Formulario de registro	Herson	—	—	—	4	—	Mixto (30 % IA)	Formulario repetitivo: la IA ayudó con la estructura de validaciones y HTML; la lógica y los ajustes fueron manuales.
RF-01	#14	Avisos y mensajes de confirmación o error del registro	Herson	—	—	—	4	—	Manual	Los avisos y mensajes requirieron ajustes en varios componentes.
RF-02	#18	Pantalla de login	Herson	—	—	—	4	—	Manual	Cambio a signals y diseño responsive del login.
RF-02	#19	Rutas Angular y guards por rol	Herson	—	—	—	4	—	Manual	El guard fue directo; las rutas y los roles requirieron pruebas.
RF-06	#74	Listado y detalle de doctores con horarios	Herson	—	—	—	5	—	Mixto (30 % IA)	CRUD y modales con apoyo de IA; permisos por rol y horarios a mano.
RF-06	#75	Editar y desactivar doctor	Herson	—	—	—	5	—	Mixto (25 % IA)	Estructura del modal con IA; permisos y confirmaciones a mano.
RF-04	#70	Listado, ficha, crear y editar paciente	Yohana	—	—	—	2	—	Manual	Pantalla del estado de los pacientes; se reutilizaron componentes y modales que ya existían en el módulo de pacientes.

En este sprint no hay estimación registrada por tarea, por lo que no se calcula desviación.

11.3.2 Sprint 4 (24 de septiembre al 8 de octubre de 2026)

Req.	Issue	Tarea	Resp.	Opt	Prob	Pes	Real	Desv	Apoyo	Causa
RF-07	#97	Esquema de citas, índice parcial y migración	Alejo	—	—	—	2	—	Manual	Montar PostgreSQL local para probar y ejecutar la migración dos veces para comprobar que se puede repetir.
RF-07	#103	GET /citas y GET /citas/{id} por rol	Alejo	—	—	—	0.5	—	Manual	Reutilizó las reglas por rol que ya existían.
RF-08	#107	PATCH /citas/{id}/estado	Alejo	—	—	—	0.5	—	Manual	Cambios de estado simples, sin validar la hora.
RF-13	#110	GET /citas/proximas	Alejo	—	—	—	2	—	Manual	Se hizo junto con #113, #114 y #118 en un solo PR (#135).
RF-14	#113	Envío programado de correo	Alejo	—	—	—	0.25	—	Manual	Spring ya trae @Scheduled y el envío de correo.
RF-14	#114	Variables SMTP en los cinco lugares	Alejo	—	—	—	0.25	—	Manual	Solo agregar las variables, pero en cinco lugares.
RF-07	#118	Datos con fechas dinámicas para las pruebas	Alejo	—	—	—	0.5	—	Manual	Preparar datos con fechas dinámicas para que las pruebas no dependan del día.
RT-05	#119	Configuración de Swagger	Carlos	—	—	—	1.5	—	Mixto	Requirió revisar compatibilidades, autenticación y funcionamiento de los endpoints.

Req.	Issue	Tarea	Resp.	Opt	Prob	Pes	Real	Desv	Apoyo	Causa
RT-05	#120	Anotaciones, permisos y respuestas HTTP	Carlos	—	—	—	2	—	Mixto	La revisión de permisos y respuestas HTTP tomó más tiempo que agregar las anotaciones.
RT-05	#121	Documentación de las operaciones de citas	Carlos	—	—	—	1	—	Mixto	Relativamente rápida; lo principal fue comprobar las operaciones de citas.
RT-04	#122	nginx y Swagger en EC2	Carlos	—	—	—	2	—	Principalmente manual	Configurar nginx y verificar Swagger desde EC2 requirió varias pruebas y ajustes.
RT-04	#123	Migración en EC2	Carlos	—	—	—	2.5	—	Principalmente manual	La revisión de migraciones y las comprobaciones en EC2 necesitaron cuidado para no afectar los datos.
RF-13	#111	Campana en el encabezado y aviso al iniciar sesión	Herson	—	—	—	5	—	Manual	Modelos y servicio; campana y notificación.
RF-04	#116	Crear paciente desde el frontend	Herson	—	—	—	5	—	Manual	Formularios reutilizando el registro; modo recepción y redirección.
RF-07	#98	Contrato de /citas	Mynor	—	—	—	1	—	Con IA	Definir el contrato antes de implementar.
RF-07	#99	Entidad Cita, enum EstadoCita y repositorio	Mynor	—	—	—	1	—	Con IA	Implementación por fases; el tiempo estuvo en definir las reglas y revisar cada fase.
RF-07	#100	Zona horaria con Clock	Mynor	—	—	—	1	—	Con IA	Hora de Guatemala y conversión a día de la semana.

Req.	Issue	Tarea	Resp.	Opt	Prob	Pes	Real	Desv	Apoyo	Causa
RF-07	#101	CitaService.crearCita con validaciones	Mynor	—	—	—	3	—	Con IA	Siete validaciones con sus pruebas unitarias.
RF-07	#102	CitaController, DTOs y reglas de seguridad	Mynor	—	—	—	2	—	Con IA	Controller, DTOs y SecurityConfig, con pruebas de integración.
RF-07	#105	Pantalla Agendar cita	Yohana	—	—	—	4	—	Manual	Formulario reactivo para agendar citas (/citas/nueva), con el selector de pacientes oculto para el rol cliente y manejo de los mensajes de error del backend.
RF-08	#108	Listado de citas con acciones de estado	Yohana	—	—	—	3.5	—	Manual	Pantalla de listado de citas (/citas), con filtros por fecha y botones para cancelar o marcar como atendida según el rol.
RF-04	#117	Desactivar paciente desde el frontend	Yohana	—	—	—	2	—	Manual	Método de desactivación en el servicio de pacientes y botón de confirmación exclusivo para administrador y secretaria.

En este sprint no hay estimación registrada por tarea, por lo que no se calcula desviación.

11.3.3 Sprint 5 (9 al 16 de octubre de 2026)

Req.	Issue	Tarea	Resp.	Opt	Prob	Pes	Real	Desv	Apoyo	Causa
RT-01	#147	Esquema del Sprint 5	Alejo	2	3	4.5	0.5	-2.5	Manual	Migración del esquema probada en local antes de subirla; esquema sencillo.

Req.	Issue	Tarea	Resp.	Opt	Prob	Pes	Real	Desv	Apoyo	Causa
RT-05	#166	Corregir N+1 en pacientes y doctores	Alejo	0.75	1	1.5	1	0	Manual	Corrección puntual de consultas.
RT-06	#172	Datos de demostración	Alejo	0.75	1	1.5	1	0	Manual	Armar datos coherentes (doctor, horarios, citas de hoy y mañana, bloqueo).
RF-14	#171	SMTP real en EC2 y cron configurable	Alejo	1	1.5	2.25	1	-0.5	Manual	Configurar SMTP real en EC2 y probar el envío; la tarea quedó marcada como duplicada.
RT-06	#173	Migración y release a EC2	Alejo	1	1.5	2.25	1.25	-0.25	Manual	Migración en EC2 y release; hubo que hacer un segundo release (PR #194).
RF-09	#151	Frontend: motivo al cancelar y marcar estados	Herson	1.75	2.5	3.75	4	+1.5	Manual	Modal de motivo, columna y manejo del 409 del motivo obligatorio.
RF-10	#154	Frontend: botón Reprogramar	Herson	1.75	2.5	3.75	5	+2.5	Mixto (30 % IA)	Servicio de reprogramar con apoyo de IA; modal y recarga de la campana a mano.
RF-11	#157	Frontend: bloqueos en el detalle del doctor	Herson	2.75	4	6	5	+1	Manual	Modelo y tarjeta; permisos y modal.
RT-05	#167	Frontend: un solo aviso en 403 y campana que se refresca	Herson	0.75	1	1.5	4	+3	Manual	Ajustes del 403, fecha local y esperas fijas en varios archivos.
RF-17	#192	Frontend: Mis tratamientos	Herson	1.5	2	3	5	+3	Manual	Agrupación por cita, diseño responsive y estados.
RT-05	#165	Swagger: reglas, errores y ejemplos	Carlos	2	3	4.5	3	0	Mixto	La revisión de reglas de negocio y errores de los endpoints aumentó el tiempo de validación.

Req.	Issue	Tarea	Resp.	Opt	Prob	Pes	Real	Desv	Apoyo	Causa
RT-06	#170	CD condicionado al CI y credenciales como secrets	Carlos	1	1.5	2.25	3	+1.5	Mixto	Configurar los secretos, comprobar el comportamiento del CI y CD y repetir las verificaciones.
RF-09	#149	Extraer la validación de disponibilidad	Mynor	1	1.5	2.25	1	-0.5	Con IA	Refactor sin cambiar comportamiento; el tiempo estuvo en las pruebas y el PR.
RF-09	#150	Motivo y plazo de 24 h	Mynor	3.5	5	7.5	3	-2	Con IA	Implementado por fases con pruebas de éxito y rechazo; el tiempo estuvo en revisar la fase y el PR.
RF-10	#153	Reprogramar cita (backend)	Mynor	3.5	5	7.5	3	-2	Con IA	Ocho reglas en orden y registro del cambio, por fases y con pruebas.
RF-11	#156	Bloqueos de doctor (backend)	Mynor	3.5	5	7.5	3	-2	Con IA	Entidad, servicio, controller y validación en la disponibilidad, con pruebas.
RF-17	#159	Tratamientos por cita (backend)	Mynor	4.25	6	9	3.5	-2.5	Con IA	Dos entidades, dos controllers y cuatro endpoints, con pruebas.
RF-20	#162	Reporte de citas por doctor y estado	Mynor	2	3	4.5	2	-1	Con IA	Una consulta agregada y pruebas con bordes de rango.
RT-06	#174	README, tablero y ramas del Sprint 5	Mynor	1	1.5	2.25	1.5	0	Manual	Documentación de las reglas nuevas y limpieza de issues y ramas.
RF-17	#160	Frontend: tratamientos en el historial del paciente	Yohana	3.5	5	7.5	3.5	-1.5	Manual	Pantalla del historial clínico para registrar tratamientos y costos, con la tabla que calcula el total acumulado del paciente.

Req.	Issue	Tarea	Resp.	Opt	Prob	Pes	Real	Desv	Apoyo	Causa
RF-20	#163	Frontend: pantalla de reportes	Yohana	2.75	4	6	4	0	Manual	Pantalla /reportes con su acceso en el sidebar, filtros por rango de fechas y tablas del reporte para administración.
RT-05	#168	Frontend: fecha mínima local y esperas fijas	Yohana	0.75	1	1.5	2.5	+1.5	Manual	Se forzó la fecha local de Guatemala en el formulario de citas y se eliminaron las esperas fijas de milisegundos.

Total de las 22 tareas con estimación y horas reportadas: probable 61.5 h, real 60.75 h, desviación -0.75 h (-1 %).

11.3.4 Sprint 6 (16 al 28 de octubre de 2026)

Req.	Issue	Tarea	Resp.	Opt	Prob	Pes	Real	Desv	Apoyo	Causa
RF-15	#196	Inasistencias: backend	Alejo	5	7	10.5	0.5	-6.5	Manual	Decidir el diseño (tabla propia y no la lista negra) y sus pruebas.
RF-18	#199	Esquema de planes y abonos	Alejo	0.75	1	1.5	0.5	-0.5	Manual	Esquema sencillo, siguió el patrón de las migraciones anteriores.
RF-21	#205	Consultas agregadas de reportes	Alejo	2	3	4.5	1	-2	Manual	Consultas agrupadas, junto con #206 (PR #232).
RF-21	#206	Consultas agregadas de reportes	Alejo	2	3	4.5	0.5	-2.5	Manual	Ídem #205.
RT-07	#210	Respaldo y restauración en EC2	Alejo	1.5	2	3	1	-1	Manual	Probar el respaldo y la restauración en EC2 comparando filas tabla por tabla.
RT-07	#211	Datos de demo del Sprint 6	Alejo	1.75	2.5	3.75	0.5	-2	Manual	Datos de demo y recorrido con el backend real para comprobarlos.

Req.	Issue	Tarea	Resp.	Opt	Prob	Pes	Real	Desv	Apoyo	Causa
RT-08	#213	Migración, release y flujo en EC2	Alejo	2.75	4	6	1	-3	Manual	La instancia estaba apagada (Learner Lab), el SSH fallaba y hubo que correr la migración otra vez.
RF-19	#201	Abonos y saldo del plan	Carlos	2.75	4	6	3	-1	Con IA	La IA agilizó la implementación; las dificultades fueron las dependencias, los pagos concurrentes y las pruebas de seguridad.
RF-21	#207	Reporte de horas más demandadas	Carlos	1.5	2	3	1.5	-0.5	Con IA	La consulta agregada y las pruebas automatizadas redujeron el tiempo de desarrollo.
RT-05	#214	Auditoría de Swagger y OpenAPI	Carlos	1.5	2	3	4	+2	Con IA	Se trabajó por etapas porque dependía de integraciones de otros compañeros; las revisiones de Swagger y las pruebas requirieron varias iteraciones.
RF-16	#223	CRUD del catálogo de tratamientos	Carlos	2.75	4	6	2.5	-1.5	Con IA	Se agilizó la implementación; hubo que validar permisos, migraciones y compatibilidad con el sistema.
RF-15	#197	Frontend: inasistencias y levantamiento	Herson	2.75	4	6	5	+1	Mixto (30 % IA)	Indicador y modal con apoyo de IA; levantamiento y manejo del 409 a mano.
RF-19	#203	Frontend: abonos y Mis planes	Herson	2.75	4	6	5	+1	Manual	Modelos y servicio de abonos; detalle del plan y pantalla Mis planes.

Req.	Issue	Tarea	Resp.	Opt	Prob	Pes	Real	Desv	Apoyo	Causa
RF-16	#224	Frontend: catálogo de tratamientos	Herson	2.75	4	6	5	+1	Mixto (30 % IA)	CRUD del catálogo con apoyo de IA; errores y permisos a mano.
RF-12	#220	Backend: bloqueos vigentes de un doctor	Mynor	1	1.5	2.25	1	-0.5	Con IA	Endpoint de lectura con pruebas unitarias y de integración.
RF-21	(sin issue)	Reporte de ganancias y reportes sin citas futuras	Mynor	—	—	—	2.5	—	Con IA	Cambio pedido durante el sprint, no planificado: dos fases adicionales sobre los reportes.
RF-12	#221	Frontend: cartel de doctor no disponible	Yohana	2	3	4.5	5	+2	Manual	Lógica para consultar los bloqueos de doctores en tiempo real y deshabilitar el botón de enviar si las fechas chocan; además, la estructura inicial del reporte de ganancias.
RF-18 y RF-21	—	Frontend: pantallas de planes y de reportes finales	Yohana	8.5	12	18	n/r	—	n/r	Sin horas reportadas. Evidencia: PR #241 (22 oct 2026, 4 commits, 23 archivos).

Total de las 16 tareas con estimación y horas reportadas: probable 51 h, real 37 h, desviación -14 h (-27 %).

11.3.5 Sprint 7 (tareas cerradas) (desde el 28 de octubre de 2026)

Req.	Issue	Tarea	Resp.	Opt	Prob	Pes	Real	Desv	Apoyo	Causa
RF-15 y RF-16	#249	Frontend: aviso de inasistencias y catálogo	Herson	2	3	4.5	4	+1	Manual	Verificación del 409 y pruebas del catálogo.
RNF	#250	Frontend: carga perezosa de rutas	Herson	1.5	2	3	4	+2	Manual	Conversión a carga perezosa y ajuste del presupuesto de tamaño.

Req.	Issue	Tarea	Resp.	Opt	Prob	Pes	Real	Desv	Apoyo	Causa
RNF	#252	Frontend: pruebas de los componentes	Herson	2.75	4	6	5	+1	Mixto (30 % IA)	Estructura de TestBed con apoyo de IA; casos de prueba a mano.
Doc.	#261	Diagrama de procesos del flujo principal	Herson	1.5	2	3	4	+2	Manual	Validación del flujo completo.
Doc.	#258	Propósito, alcance y requisitos funcionales	Mynor	2.75	4	6	2.5	-1.5	Con IA	Borrador con apoyo de IA; el tiempo estuvo en comprobar cada requisito contra el backlog.
Doc.	#260	Diagramas de arquitectura y de despliegue	Mynor	1.75	2.5	3.75	1.5	-1	Manual	Diagramas generados y luego revisados y exportados.
Doc.	#266	Historial de sprints y retrospectiva	Mynor	2.75	4	6	3	-1	Manual	Borrador con la evidencia de git; el tiempo estuvo en revisarlo y corregirlo.
RF-21	#247	Frontend: pantalla de ganancias en reportes	Yohana	2	3	4.5	3.5	+0.5	Manual	Cierre del frontend de reportes: la pantalla de ganancias se unificó en el selector, se controlaron los errores 400 y se separaron los totales de tratamientos y abonos.

Total de las 8 tareas con estimación y horas reportadas: probable 24.5 h, real 27.5 h, desviación +3 h (+12 %).

11.4 Evidencia de git

Pull requests de trabajo fusionados por integrante entre el 6 de agosto y el 28 de octubre de 2026, sin contar los 9 releases de develop a main ni el primer release revertido:

Integrante	Pull requests	Observación
Carlos	22	Infraestructura, CI y CD, Swagger y parte del backend.
Luis	21	Base de datos y backend.
Mynor	17	Citas, reglas de negocio y coordinación del backlog.
Herson	16	Frontend.
Yohana	5	Frontend; pocos pull requests pero con muchos commits (el #55 tiene 8 y el #241, 4).
Compartido	1	El #185 reúne 3 commits de Yohana y 2 de Herson.
Total	82	

La cantidad de pull requests no mide el esfuerzo: el trabajo del frontend se agrupó en pocos pull requests con muchos archivos. Las horas de las pantallas de planes y de reportes finales (issues #202 y #208) no fueron reportadas, por lo que esas tareas aparecen con su evidencia de entrega y sin horas.

11.5 Análisis de desviaciones

En los sprints 5 a 7 el total estimado y el real se parecen en el Sprint 5 y en el Sprint 7 (61.5 h frente a 60.75 h y 24.5 h frente a 27.5 h) y difieren en el Sprint 6 (51 h frente a 37 h), pero esas cifras ocultan desviaciones de signo contrario según el tipo de trabajo:

- **Backend e infraestructura (-42 %).** Las tareas con patrones repetidos, como las migraciones, las consultas agregadas de los reportes y los endpoints de estructura similar, tomaron menos de lo estimado. La estimación se hizo pensando en escribir todo a mano y no consideró el apoyo de IA ni la reutilización de reglas ya existentes, como el servicio de disponibilidad de citas.
- **Frontend (+42 %).** Fue el único tipo de trabajo que superó la estimación en los tres sprints (Sprint 5: 33 h frente a 22; Sprint 6: 20 h frente a 15; Sprint 7: 16.5 h frente a 12). Las tareas incluyeron ajustes en varios archivos, diseño responsive, manejo de los errores 409 y de los permisos por rol, y pruebas, que no estaban en la estimación.
- **Infraestructura con dependencias externas.** Las tareas que dependían de otras personas o de servicios externos se alargaron: la configuración de secretos y las verificaciones del

CD (#170, 3 h frente a 1.5), la auditoría de Swagger que dependía de otros pull requests (#214, 4 h frente a 2) y el despliegue en EC2, donde la instancia estaba apagada (Learner Lab) y hubo que repetir la migración y hacer un segundo release.

- **Trabajo no planificado.** Durante los sprints aparecieron cambios que no tenían estimación: la pantalla Mis tratamientos del paciente, el reporte de ganancias, la regla de reportes sin citas futuras y el endpoint de bloqueos vigentes.
- **Falta de registro.** Antes del Sprint 5 no se conservaron estimaciones por tarea y en ningún sprint se registraron horas reales en el tablero, por lo que la comparación depende de reportes aproximados.

11.6 Acciones para estimar mejor

- **Aplicar un factor al frontend.** Mientras no haya más datos, estimar el frontend con un factor de 1.4 sobre la estimación base, porque en los tres sprints superó lo estimado (entre 1.3 y 1.5 veces).
- **Estimar el backend por patrón.** Separar las tareas repetitivas (migraciones, consultas agregadas, endpoints similares) de las que definen reglas nuevas, y asignar a las primeras un rango menor, anotando si se harán con o sin apoyo de IA.
- **Incluir la integración en la estimación.** Sumar el tiempo del pull request, la aprobación, los conflictos y el despliegue, que fue lo que alargó las tareas de infraestructura.
- **Usar la estimación pesimista en tareas con dependencias externas.** Para trabajo que depende de EC2, de secretos o de otros compañeros, planificar con la estimación pesimista.
- **Registrar horas desde el primer sprint.** Anotar en el tablero la estimación y las horas reales de cada issue al cerrarlo, en lugar de reconstruirlas al final.
- **Reservar capacidad para cambios del Product Owner.** Dejar una parte del sprint, por ejemplo el 15 %, para cambios pedidos durante el sprint.